Name of two GenAI used: ChatGPT and Gemini AI
How they were accessed: Web browser on personal computer (chatgpt.com, gemini.google.com)
Prompt used: create this code: Assignment 3 instructions.pdf
Code generated by both GenAI:

Gemini:

```
/*
 * Program Name: EECS 348 Assignment 3 - Email Priority Queue
 * Description: Implements an object-oriented CEO email priority system using a
 *              custom list-based MaxHeap data structure (using std::vector as a
 *              dynamic list container without STL heap algorithms). Parses EMAIL,
 *              COUNT, NEXT, and READ commands from standard input.
 * Inputs: Standard input stream containing formatted commands (EMAIL, NEXT, READ, COUNT).
 * Output: Terminal output displaying current unread email counts and email previews.
 *
 * Author: [Your Full Name]
 * Creation Date: September 29, 2026
 * Revision Date: September 29, 2026
 * Revisions: Refined AI-generated output to implement object-oriented structure,
 *            list-based MaxHeap logic, precise date parsing, and robust edge cases.
 *
 * Collaborators: None
 * Other Sources: Code initial skeleton generated via ChatGPT and Claude, modified by author.
 */

#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <algorithm>

// ============================================================================
// Class: Date
// Author: Combination of ChatGPT and Student
// Description: Represents a date (MM-DD-YYYY) and provides comparison logic
//              to support date-based prioritization.
// ============================================================================
class Date {
private:
    int month; // Holds integer month (1-12)
    int day;   // Holds integer day (1-31)
    int year;  // Holds integer year (e.g., 2024, 2025)

public:
    // Default constructor initialized to zero
    Date() : month(0), day(0), year(0) {}

    // Parameterized constructor parsing string MM-DD-YYYY
    Date(const std::string& dateStr) {
        // Author: Student refinement for safe string parsing
        if (dateStr.length() >= 10) {
            month = std::stoi(dateStr.substr(0, 2)); // Extract month
            day = std::stoi(dateStr.substr(3, 2));   // Extract day
```

```
            year = std::stoi(dateStr.substr(6, 4));  // Extract year
        } else {
            month = day = year = 0; // Fallback for invalid input format
        }
    }

    // Returns full formatted date string
    std::string toString() const {
        // Formats back to string format MM-DD-YYYY
        char buf[12];
        snprintf(buf, sizeof(buf), "%02d-%02d-%04d", month, day, year);
        return std::string(buf);
    }

    // Comparison operator to determine if this date is NEWER than another date
    bool operator>(const Date& other) const {
        // Compare years first
        if (year != other.year) return year > other.year;
        // Compare months if years are equal
        if (month != other.month) return month > other.month;
        // Compare days if months and years are equal
        return day > other.day;
    }
};

// ================================================================================
// Class: Email
// Author: Combination of Claude and Student
// Description: Encapsulates email attributes and comparison logic for priority.
// ================================================================================
class Email {
private:
    std::string senderCategory; // Category: Boss, Subordinate, Peer, etc.
    std::string subject;        // Subject line string
    std::string rawDate;        // Raw string date
    Date dateObj;               // Parsed Date object for comparison

    // Helper method to convert category string to integer rank
    int getCategoryPriority() const {
        // Boss read first (highest rank = 5)
        if (senderCategory == "Boss") return 5;
        // Subordinate read next (rank = 4)
        if (senderCategory == "Subordinate") return 4;
        // Peer read next (rank = 3)
        if (senderCategory == "Peer") return 3;
        // Important Person read next (rank = 2)
        if (senderCategory == "Important Person" || senderCategory == "ImportantPerson") return 2;
        // Other Person read last (rank = 1)
        return 1;
    }

public:
    // Constructor initializing empty email
    Email() = default;
```

```
    // Parameterized constructor
    Email(const std::string& cat, const std::string& subj, const std::string& d)
        : senderCategory(cat), subject(subj), rawDate(d), dateObj(d) {}

    // Getters for email fields
    std::string getSender() const { return senderCategory; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return rawDate; }

    // Operator to compare email priorities (returns true if this has HIGHER priority than other)
    bool operator>(const Email& other) const {
        int p1 = getCategoryPriority();
        int p2 = other.getCategoryPriority();

        // Higher category priority comes first
        if (p1 != p2) {
            return p1 > p2;
        }

        // If same category, newest email (not oldest) comes first
        return dateObj > other.dateObj;
    }
};

// ============================================================================
// Class: MaxHeap
// Author: Student (refactored from AI base logic)
// Description: Custom list-based implementation of a Max-Heap priority queue.
// ============================================================================
class MaxHeap {
private:
    std::vector<Email> heapList; // Underlying dynamic list storage for binary heap nodes

    // Restores heap property upward from given index
    void bubbleUp(size_t index) {
        // Loop while not at root and current node > parent node
        while (index > 0) {
            size_t parentIndex = (index - 1) / 2; // Compute parent index
            if (heapList[index] > heapList[parentIndex]) {
                std::swap(heapList[index], heapList[parentIndex]); // Swap parent and child
                index = parentIndex; // Move index to parent position
            } else {
                break; // Heap property satisfied
            }
        }
    }

    // Restores heap property downward from given index
    void bubbleDown(size_t index) {
        size_t size = heapList.size(); // Current heap size
        while (index < size) {
            size_t leftChild = 2 * index + 1;  // Left child index
            size_t rightChild = 2 * index + 2; // Right child index
```

```
            size_t largest = index;           // Initialize largest as root

            // Check if left child exists and is greater than current largest
            if (leftChild < size && heapList[leftChild] > heapList[largest]) {
                largest = leftChild;
            }

            // Check if right child exists and is greater than current largest
            if (rightChild < size && heapList[rightChild] > heapList[largest]) {
                largest = rightChild;
            }

            // If largest is not root, swap and continue bubbling down
            if (largest != index) {
                std::swap(heapList[index], heapList[largest]);
                index = largest;
            } else {
                break; // Heap property satisfied
            }
        }
    }

public:
    // Inserts a new email into the priority heap
    void insert(const Email& email) {
        heapList.push_back(email);     // Add new element to end of list
        bubbleUp(heapList.size() - 1); // Bubble up new element to maintain MaxHeap property
    }

    // Removes and returns highest-priority email
    bool removeMax() {
        if (isEmpty()) return false; // Handle empty heap gracefully
        heapList[0] = heapList.back(); // Overwrite root with last element
        heapList.pop_back();           // Remove last element
        if (!isEmpty()) {
            bubbleDown(0);             // Restore heap property from root
        }
        return true;
    }

    // Returns pointer to top email without removing it
    const Email* peekMax() const {
        if (isEmpty()) return nullptr; // Return nullptr if empty
        return &heapList[0];           // Return reference to root element
    }

    // Returns total count of items in heap
    size_t size() const {
        return heapList.size(); // Return size of vector list
    }

    // Checks if heap is empty
    bool isEmpty() const {
        return heapList.empty(); // Return true if empty
```

```
    }
};

// Helper function to trim leading and trailing spaces from strings
std::string trim(const std::string& str) {
    size_t start = str.find_first_not_of(" \t\r\n");
    if (start == std::string::npos) return "";
    size_t end = str.find_last_not_of(" \t\r\n");
    return str.substr(start, end - start + 1);
}

// ================================================================================
// Main Execution Engine
// Author: Student
// Description: Handles stream parsing, email extraction, and heap command processing.
// ================================================================================
int main() {
    MaxHeap priorityQueue; // Instantiate custom MaxHeap object
    std::string line;       // Input line buffer

    // Read commands line by line until EOF
    while (std::getline(std::cin, line)) {
        line = trim(line); // Clean up whitespace around command line
        if (line.empty()) continue; // Skip empty lines

        // Command processing logic
        if (line.rfind("EMAIL", 0) == 0) {
            // Process EMAIL command
            std::string payload = line.substr(5); // Strip "EMAIL" prefix
            std::stringstream ss(payload);
            std::string category, subject, dateStr;

            // Parse comma-delimited fields: <sender category>, <subject line>, <date>
            if (std::getline(ss, category, ',') &&
                std::getline(ss, subject, ',') &&
                std::getline(ss, dateStr)) {

                // Clean leading/trailing spaces from extracted tokens
                category = trim(category);
                subject = trim(subject);
                dateStr = trim(dateStr);

                // Create email object and insert into MaxHeap
                Email newEmail(category, subject, dateStr);
                priorityQueue.insert(newEmail);
            }
        }
        else if (line == "COUNT") {
            // Display current count of unread emails
            std::cout << "There are " << priorityQueue.size() << " emails to read." << std::endl;
        }
        else if (line == "NEXT") {
            // Preview highest-priority email without deleting it
            const Email* topEmail = priorityQueue.peekMax();
```

```
            if (topEmail != nullptr) {
                std::cout << "Next email:" << std::endl;
                std::cout << "Sender: " << topEmail->getSender() << std::endl;
                std::cout << "Subject: " << topEmail->getSubject() << std::endl;
                std::cout << "Date: " << topEmail->getDate() << std::endl;
            }
        }
        else if (line == "READ") {
            // CEO has read email; delete highest priority email from queue
            priorityQueue.removeMax();
        }
    }

    return 0; // Return success status
}
```

ChatGPT:

```
/*
 * Program: EECS 348 Assignment 3
 * Description:
 *   This program manages and prioritizes emails for a CEO.
 *   A MaxHeap is implemented from scratch using a vector.
 *   Emails are prioritized first by sender category and then
 *   by date, with newer emails receiving higher priority.
 *
 * Input:
 *   A text file containing EMAIL, NEXT, READ, and COUNT commands.
 *
 * Output:
 *   Displays the highest-priority email and the number of
 *   unread emails when requested.
 *
 * Collaborators: None
 * Other Sources: ChatGPT
 * Author: [YOUR FULL NAME]
 * Creation Date: September 29, 2026
 * Revision Date: September 29, 2026
 * Revisions: Initial version
 */

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

/*
 * Email class
 * Stores the sender category, subject, and date for one email.
 * This class/block was generated with assistance from ChatGPT.
 */
class Email
{
```

```
private:
   string sender;          // Stores the sender category.
   string subject;         // Stores the email subject.
   string date;            // Stores the original date string.
   int dateValue;          // Stores a numeric version of the date for comparison.

public:
   // Default constructor creates an empty email.
   Email()
   {
      sender = "";        // Initialize sender.
      subject = "";       // Initialize subject.
      date = "";          // Initialize date.
      dateValue = 0;      // Initialize numeric date.
   }

   // Constructor creates an email using information read from the file.
   Email(string newSender, string newSubject, string newDate)
   {
      sender = newSender;             // Save sender category.
      subject = newSubject;           // Save subject.
      date = newDate;                 // Save original date.
      dateValue = convertDate(date);   // Convert date for comparisons.
   }

   // Converts MM-DD-YYYY into YYYYMMDD so dates can be compared numerically.
   int convertDate(string emailDate)
   {
      int month;                    // Stores month.
      int day;                      // Stores day.
      int year;                     // Stores year.
      char dash1;                   // Stores first '-'.
      char dash2;                   // Stores second '-'.

      stringstream ss(emailDate);      // Create stream from date.
      ss >> month >> dash1 >> day >> dash2 >> year; // Extract date parts.

      return year * 10000 + month * 100 + day; // Return sortable date value.
   }

   // Returns the priority number associated with the sender category.
   int getSenderPriority() const
   {
      if (sender == "Boss")           // Boss has highest priority.
      {
         return 5;
      }
      else if (sender == "Subordinate") // Subordinate is second.
      {
         return 4;
      }
      else if (sender == "Peer")       // Peer is third.
      {
         return 3;
```

```
        }
        else if (sender == "ImportantPerson") // ImportantPerson is fourth.
        {
            return 2;
        }
        else                          // OtherPerson has lowest priority.
        {
            return 1;
        }
    }

    // Determines whether this email has higher priority than another email.
    bool higherPriorityThan(const Email& other) const
    {
        if (getSenderPriority() != other.getSenderPriority())
        {
            return getSenderPriority() > other.getSenderPriority();
        }

        // If categories are equal, the newest email gets priority.
        return dateValue > other.dateValue;
    }

    // Displays the email using the assignment's required format.
    void display() const
    {
        cout << "Sender: " << sender << endl;   // Display sender.
        cout << "Subject: " << subject << endl; // Display subject.
        cout << "Date: " << date << endl;       // Display date.
    }
};

/*
 * MaxHeap class
 * Implements a MaxHeap from scratch using a vector.
 * This class/block was generated with assistance from ChatGPT.
 */
class MaxHeap
{
private:
    vector<Email> heap; // List-based storage for the MaxHeap.

    // Moves an email upward until the MaxHeap property is restored.
    void heapifyUp(int index)
    {
        while (index > 0)
        {
            int parent = (index - 1) / 2; // Calculate parent index.

            if (heap[index].higherPriorityThan(heap[parent]))
            {
                Email temp = heap[index]; // Temporarily save child.
                heap[index] = heap[parent]; // Move parent downward.
                heap[parent] = temp;      // Move child upward.
```

```
            index = parent;          // Continue from new position.
        }
        else
        {
            break;                   // Heap property has been restored.
        }
    }
}

// Moves an email downward until the MaxHeap property is restored.
void heapifyDown(int index)
{
    int size = heap.size();          // Store current heap size.

    while (true)
    {
        int left = 2 * index + 1;    // Calculate left child.
        int right = 2 * index + 2;   // Calculate right child.
        int largest = index;         // Assume current item has highest priority.

        if (left < size &&
            heap[left].higherPriorityThan(heap[largest]))
        {
            largest = left;          // Left child has higher priority.
        }

        if (right < size &&
            heap[right].higherPriorityThan(heap[largest]))
        {
            largest = right;         // Right child has higher priority.
        }

        if (largest == index)
        {
            break;                   // Heap property is correct.
        }

        Email temp = heap[index];    // Temporarily save current email.
        heap[index] = heap[largest]; // Move higher-priority child upward.
        heap[largest] = temp;        // Move current email downward.

        index = largest;             // Continue checking new position.
    }
}

public:
// Adds an email to the MaxHeap.
void insert(const Email& email)
{
    heap.push_back(email);           // Add email to end of list.
    heapifyUp(heap.size() - 1);      // Restore MaxHeap property.
}
```

```
    // Displays the highest-priority email without removing it.
    void next() const
    {
        if (heap.empty())               // Check for an empty inbox.
        {
            return;                     // Nothing can be displayed.
        }

        cout << "Next email:" << endl;    // Required NEXT heading.
        heap[0].display();              // Root contains highest-priority email.
    }

    // Removes the highest-priority email.
    void read()
    {
        if (heap.empty())               // Check for an empty inbox.
        {
            return;                     // Nothing can be removed.
        }

        heap[0] = heap.back();           // Move final email to root.
        heap.pop_back();                 // Remove final position.

        if (!heap.empty())              // Check whether emails remain.
        {
            heapifyDown(0);              // Restore MaxHeap property.
        }
    }

    // Displays the number of unread emails.
    void count() const
    {
        cout << "There are " << heap.size()
             << " emails to read." << endl;
    }
};

/*
 * EmailManager class
 * Handles reading the command file and interacting with the MaxHeap.
 * This class/block was generated with assistance from ChatGPT.
 */
class EmailManager
{
private:
    MaxHeap inbox; // CEO's priority inbox.

    // Processes an EMAIL command.
    void processEmail(string line)
    {
        // Remove "EMAIL " from the beginning.
        string emailData = line.substr(6);

        // Locate the commas separating the three fields.
```

```
        size_t firstComma = emailData.find(',');
        size_t secondComma = emailData.find(',', firstComma + 1);

        // Extract sender category.
        string sender = emailData.substr(0, firstComma);

        // Extract subject, which may contain spaces.
        string subject = emailData.substr(
            firstComma + 1,
            secondComma - firstComma - 1
        );

        // Extract date.
        string date = emailData.substr(secondComma + 1);

        // Create an Email object.
        Email newEmail(sender, subject, date);

        // Insert it into the priority queue.
        inbox.insert(newEmail);
    }

public:
    // Reads and processes every command in the specified file.
    void run(string filename)
    {
        ifstream inputFile(filename);      // Open command file.

        if (!inputFile.is_open())         // Verify that the file opened.
        {
            cout << "Error opening file." << endl;
            return;
        }

        string line;                      // Stores each input line.

        while (getline(inputFile, line))  // Read file one line at a time.
        {
            if (line.rfind("EMAIL ", 0) == 0)
            {
                processEmail(line);       // Add new email.
            }
            else if (line == "NEXT")
            {
                inbox.next();             // Display highest-priority email.
            }
            else if (line == "READ")
            {
                inbox.read();             // Remove highest-priority email.
            }
            else if (line == "COUNT")
            {
                inbox.count();            // Display unread count.
            }
```

```
        }

        inputFile.close();              // Close input file.
    }
};

/*
 * Main program
 * Creates the EmailManager object and starts processing the input file.
 * This block was generated with assistance from ChatGPT.
 */
int main(int argc, char* argv[])
{
    // Require the test filename as a command-line argument.
    if (argc != 2)
    {
        cout << "Usage: " << argv[0] << " <test file>" << endl;
        return 1;
    }

    EmailManager manager;               // Create email manager object.
    manager.run(argv[1]);               // Process the supplied test file.

    return 0;                           // End program successfully.
}
```

Code Analysis

Correctness: Both programs correctly follow the assignment requirements and work with the given test input. They both correctly prioritize emails based on sender and date and handle edge cases like an empty queue, multiple NEXT commands, and multiple READ commands. Gemini does have slightly better input handling because it trims extra spaces and checks the parsed fields before adding an email.

Execution Time: Both AI generated codes have nearly identical execution times. Inserting and reading an email are both O(log n), while NEXT and COUNT are O(1). Both use vectors, so they may occasionally take O(n) when the vector needs to resize.

Space Complexity: Both programs have a space complexity of O(n) because their vectors grow depending on the number of emails. Gemini stores the date using a separate Date object while ChatGPT stores it as an integer, but the overall space complexity is still the same.

Maintainability: Both programs are easy to maintain because they separate the program into different classes. However, I think ChatGPT's code is slightly easier to follow because it separates the Email, MaxHeap, and EmailManager into their own classes, keeping the heap and command processing separate.

Choice of Code: I am personally going to choose ChatGPT's code because both programs are very similar in correctness, execution time, and space complexity, but I find ChatGPT's organization easier to understand. Separating the command processing into the EmailManager class also makes it easier to change or debug.

Improvements: I would improve ChatGPT's code by adding some of Gemini's input handling, such as checking that the commas exist and removing extra spaces from the input. This would improve the correctness and maintainability without changing the O(log n) heap operations or O(n) space complexity.